

AFDPS for Full Waveform Inversion -- Setup & Run Guide

This guide takes you from a fresh GPU box to a first benchmark number and a full tuning sweep for the AFDPS x InverseBench full waveform inversion (FWI) port. AFDPS (Approximation-Free Diffusion Posterior Sampling, TMLR 2026) is applied to the InverseBench FWI benchmark (ICLR 2025) so its reconstruction can be compared directly against the methods in InverseBench Table 7. The code is built and CPU-verified; the remaining work is the GPU + Devito setup, asset download, and running the 10-case benchmark.

GitHub: <https://github.com/erichou1/AFDPS> (public -- clone it on the GPU box). The FWI port lives in the `full_waveform_inversion/` folder; it REUSES the InverseBench harness in `navier_stokes/` as a library (same forward operator, dataset loader, evaluator and EDM prior), so every run executes with the working directory set to `navier_stokes/`. The provided scripts do that for you.

Goal: place AFDPS alongside the InverseBench Table-7 baselines (DPS, LGD, DiffPIR, DAPS, PnP-DM, REDDiff; plus Adam/LBFGS) on the SAME forward model, the SAME 10-case CurveFaultB test set, the SAME noise-free setting, and the SAME eval.AcousticWave evaluator.

Where things stand (what is already done)

- FWI operator wired: `afdns_fwi/operator.py` subclasses the benchmark Devito AcousticWave, exposing the AFDPS operator API (gradient = adjoint-state $J^T r$; Tr-Hessian estimator).
- Vendored AFDPS sampler (`afdns_fwi/sampler.py`) -- the annealed-SDE Feynman-Kac ensemble sampler, byte-identical to the Navier-Stokes port.
- CPU smoke test PASSES with no Devito/GPU/checkpoint/data: Hutchinson trace matches the exact linear-operator value to ~0.4%, the sampler reduces data misfit ~114.7 -> ~20.7, and the wrapper reductions (best/mean) produce correct shapes. Run: `python full_waveform_inversion/tests/test_sampler_cpu.py`
- Configs, multi-GPU/sharded/GB200 scripts, and a Table-7 aggregator are in place.

So the next steps are purely: get it on the GPU box, install Devito, download the FWI checkpoint + data, verify, then run the 10-case benchmark and aggregate.

Prerequisites

- A Linux GPU box (the GB200s). NVIDIA driver + CUDA runtime matching your torch build.
 - Devito (the FWI wave-equation solver runs on CPU via Devito's compiled C kernels + dask). This is the key extra dependency the Navier-Stokes port did NOT need.
 - Python 3.9+ (3.10/3.11 recommended). A C compiler for Devito (gcc/clang; usually present).
 - ~5-10 GB disk for the checkpoint + datasets. Internet access to GitHub + CaltechDATA.
-

Step 1 -- Get the repo onto the GPU box (git clone)

The project is on GitHub (public), so you do NOT copy anything from your laptop. On the GPU box, clone it:

```
git clone https://github.com/erichou1/AFDPS.git
cd AFDPS
```

This gives you the whole project, including `navier_stokes/` (the reused InverseBench harness) and `full_waveform_inversion/` (this FWI port: `afdns_fwi/`, `configs/`, `scripts/`, `tests/`, `main.py`, `SETUP_GUIDE.pdf`). Large assets are NOT in git (.gitignore excludes checkpoints/ and data/) -- you download those in Step 3. To pull later updates: `git pull`.

Step 2 -- Python environment

You need (a) the InverseBench harness deps and (b) Devito for the FWI solver.

a) Harness dependencies (same as the Navier-Stokes port)

```
python -m venv .venv && source .venv/bin/activate
pip install --upgrade pip
# CUDA build of torch matching your driver, then the rest:
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124
pip install hydra-core omegaconf lmbd numpy tqdm piq pyyaml requests
```

b) Devito + the FWI solver stack

The FWI forward/adjoint solves use Devito and a dask LocalCluster, plus Devito's seismic examples (examples.seismic). Install Devito and dask:

```
pip install devito dask distributed
```

The operator imports 'from examples.seismic import ...' (Devito's seismic example models). Those ship with Devito; if the import fails, install Devito from source or add its examples to PYTHONPATH (see navier_stokes/README_InverseBench.md and env.yaml, which pin the exact FWI solver stack the benchmark used).

Sanity-check torch sees the GPU and Devito imports:

```
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
python -c "import devito; from examples.seismic.acoustic import AcousticWaveSolver;
print('devito ok', devito.__version__)"
```

Step 3 -- Download the assets (checkpoint + data)

FWI needs TWO things the configs expect at fixed paths (relative to navier_stokes/):

- the EDM diffusion prior checkpoints/fwi-5m.pt
- the CurveFaultB test/val data ../data/fwi-test and ../data/fwi-val (LMBD dirs, a SIBLING of the repo's navier_stokes/ folder).

NOTE: navier_stokes/scripts/download_assets.sh fetches the Navier-Stokes assets only, so use the manual commands below for FWI. The checkpoint is an InverseBench GitHub release asset; the data is in the same CaltechDATA record as the NS data (FILES are restricted -> a share token is needed; reuse the token already in navier_stokes/README_InverseBench.md).

```
cd navier_stokes
mkdir -p checkpoints ../data
# 1) diffusion prior (FWI):
curl -L -o checkpoints/fwi-5m.pt \
  https://github.com/devzhk/InverseBench/releases/download/diffusion-prior/fwi-5m.pt
# 2) data (TOK = the share token from README_InverseBench.md):
B=https://data.caltech.edu/api/records/zg89b-mpv16/files
curl -L -o fwi-test.zip "$B/fwi-test.zip/content?token=$TOK"
curl -L -o fwi-val.zip "$B/fwi-val.zip/content?token=$TOK"
unzip -o fwi-test.zip -d ../data && unzip -o fwi-val.zip -d ../data
```

If a filename 404s, open the CaltechDATA record page (linked in README_InverseBench.md) and copy the exact FWI asset names; if the data returns 401/403 the token expired -- get a fresh share link from the record page. Verify the prior matches the config: problem.prior=checkpoints/fwi-5m.pt and problem.data.root=../data/fwi-test in full_waveform_inversion/configs/problem/fwi-afdp.yaml.

Step 4 -- Verify the install (no Devito / GPU / checkpoint needed)

Run the CPU smoke test first. It exercises the vendored sampler, the AFDPS inference-wrapper reduction, and the finite-difference Hutchinson trace math (against a linear mock operator with a known exact Hessian trace) -- all without Devito, a GPU, the checkpoint or the data.

```
python full_waveform_inversion/tests/test_sampler_cpu.py
# expect: trace max-rel-err ~0.004 ; misfit init ~114.7 -> best ~20.7 ; 'All CPU smoke tests
passed.'
```

This proves the AFDPS math/wiring is correct independent of the heavy FWI stack, so any later issue is isolated to Devito/GPU/assets, not the algorithm.

Step 5 -- Validate the Devito forward operator at 128x128 (CFL sanity)

Before the full run, confirm the wave solver is stable (CFL-satisfying) and the operator API is live on this box. From `navier_stokes/` (so imports + the dask cluster initialize as in a real run):

```
cd navier_stokes
python -c "import sys; sys.path[:0]=['../full_waveform_inversion','.'];\
import torch; from afdps_fwi.operator import AFDPSAcousticWave;\
op=AFDPSAcousticWave(shape=[128,128], spacing=[20.0,10.0], tn=1000.0, f0=0.005, dt=1.0,\
    nbl=80, nshots=16, nreceivers=129, src_depth=1270.0, unnorm_scale=1.0, unnorm_shift=3.0,\
    sigma_noise=1.0, device='cpu');\
x=torch.zeros(1,1,128,128); y=op.forward(x);\
print('finite:', torch.isfinite(y).all().item(), 'obs shape:', tuple(y.shape))"
```

Expect finite: True and obs shape (1, 16, T, 129). If it diverges (NaN), the velocity left the CFL-safe band -- the operator clamps velocity to `[vel_min_kms, vel_max_kms]=[1.5,4.5]` before every solve, so tighten that band or check dt. (This first call also compiles the Devito kernels, so it is slow once, then cached.)

Step 6 -- First Track run (one case, fast)

Run a SINGLE case first to get a number fast (the full 10-case run is hours). The script sets `cwd=navier_stokes/` and invokes the FWI `main.py` with the right config groups:

```
bash full_waveform_inversion/scripts/run_fwi_afdps_single.sh first 1-1
```

A live progress bar shows steps + ETA; it finishes printing 'Final metric results: {relative l2: ..., psnr: ..., ssim: ..., data misfit: ...}'. Results save under `full_waveform_inversion/results/fwi-afdps/AFDPS/first/`. A NaN or relative-L2 >> 1 almost always means the likelihood temperature / guidance is off -- go to Step 7.

Equivalent explicit command (what the script runs)

```
cd navier_stokes
python ../full_waveform_inversion/main.py \
    problem=fwi-afdps algorithm=afdps pretrain=fwi \
    problem.data.id_list=1-1 num_samples=1 wandb=false exp_name=first
```

Step 7 -- Tune (`sigma_y` is the dominant knob; FWI is noise-free)

Because the InverseBench FWI measurement is NOISE-FREE, `sigma_y` is NOT a physical noise level -- it is the likelihood TEMPERATURE (regularization strength), and it is the single most important knob. The paper search range is `[1e-2, 1e1]`. Tune it FIRST, fast (1 case + few steps). It is set by `problem.model.sigma_noise`.

```
cd navier_stokes
for s in 0.1 0.3 1.0 3.0; do \
    python ../full_waveform_inversion/main.py problem=fwi-afdps algorithm=afdps pretrain=fwi \
    problem.data.id_list=1-1 algorithm.method.num_steps=40 \
    algorithm.method.num_particles=8 problem.model.sigma_noise=$s \
    num_samples=1 wandb=false exp_name=$s ; done
```

Pick the `s` with the lowest 'relative l2', then zoom around it. Next, tune the guidance strength `guidance_gamma` (smaller -> stronger/earlier data fit; too large -> ignores data).

Other useful knobs (all overridable on the command line):

- `algorithm.method.guidance_gamma` (e.g. 0.5/1/2/5) -- annealed guidance strength.

- algorithm.method.num_steps (100/200/400) -- more steps = more stable, slower (each step is real wave solves, so this directly costs time).
- algorithm.method.num_particles (8/16/32/48) -- ensemble size; raise to fill the GPU.
- algorithm.method.reduce = best | mean -- top-weight MAP particle vs Feynman-Kac posterior mean.
- algorithm.method.sampler_kwargs.resample=true -- enable AFDPS Algorithm 1 ESS resampling (culls degenerate particles; raise resample_threshold for more frequent resampling).
- algorithm.method.sampler_kwargs.use_value=true -- add the real misfit $\mu_y(x)$ to the Feynman-Kac weight so particles are scored by actual data fit.
- problem.model.laplacian_mode = fd_divergence (default, exact trace) | gn_hutchinson (Gauss-Newton $\|J\|_F^2$, PSD/cheaper, VALIDATE on-device) | zero (drop curvature; ablation).
- problem.model.hutchinson_M (1/2/4) and hutchinson_scheme=forward|central -- trace probes / cost.
- problem.model.vel_min_kms / vel_max_kms -- CFL clamp band (widen cautiously).

Step 8 -- Full 10-case benchmark + Table-7 aggregation

Run all 10 CurveFaultB test cases. Shard them across your GPUs with the launcher (it sets cwd, splits the case ids per process, waits, and aggregates). Because main.py seeds PER GLOBAL CASE ID, the split is bit-for-bit identical to a single-GPU run -- only faster:

```
bash full_waveform_inversion/scripts/run_fwi_afdps_sharded.sh final 4 10 \
  problem.model.sigma_noise=<best_s> algorithm.method.guidance_gamma=<best_gamma>
```

It prints the InverseBench Table-7 row for AFDPS (Relative L2, PSNR, SSIM, Data misfit; mean (std) over the 10 cases) next to the published baselines. Re-aggregate any time without rerunning:

```
python full_waveform_inversion/scripts/aggregate_fwi_afdps_results.py \
  "full_waveform_inversion/results/fwi-afdps/AFDPS/final_shard*/result_*.pt" \
  --logs "navier_stokes/final_shard*.log" --label "AFDPS (final)"
```

- Resume after a crash: each finished case is saved as result_<id>.pt; rerun only the missing id_list, then re-aggregate over all result_*.pt.
- Single-GPU full run instead: bash full_waveform_inversion/scripts/run_fwi_afdps_single.sh final 1-10 problem.model.sigma_noise=<best_s>.

Step 9 -- GB200 efficiency run

FWI's bottleneck is the CPU, not the GPU: the wave forward/adjoint solves run on CPU via Devito + dask, while the GPU runs the EDM prior and the Hutchinson/weight algebra. The GB200 script drives BOTH -- a large particle ensemble on the GPU (tf32 + torch.compile) and Devito OpenMP across the Grace cores with the 16 shots parallelized by dask:

```
bash full_waveform_inversion/scripts/run_fwi_afdps_gb200.sh gb200 48 200 \
  problem.model.sigma_noise=<best_s> algorithm.method.guidance_gamma=<best_gamma>
```

- It exports DEVITO_LANGUAGE=openmp and OMP_NUM_THREADS=\$(nproc); static solver/geometry tensors are built once in the operator and reused.
- To keep both processors busy, raise num_particles (GPU) and ensure dask has the cores (OMP_NUM_THREADS) for the shots. Lower laplacian cost with hutchinson_scheme=forward or laplacian_mode=gn_hutchinson if the trace dominates.
- '100% GPU' is NOT the right target for FWI (the solver is CPU-Devito); the goal is to keep the GPU prior step and the CPU solver overlapped and to parallelize shots + cases.

Benchmark settings (must match InverseBench exactly)

- Forward: acoustic wave eq, 128x128 mesh, 2.54km x 1.27km, spacing 20m x 10m, dt=1ms (CFL), 5Hz Ricker ($f_0=0.005$), 1s record ($t_n=1000$ ms), free-surface top BC, absorbing elsewhere, nbl=80.

- Geometry: 16 sources @ 1270m depth, 129 receivers @ 10m depth.
- Data: CurveFaultB, 128x128, NOISE-FREE; 10 test cases (id_list 1-10). Normalization std=500, mean=3000 (m/s); operator unnorm_shift=3.0, unnorm_scale=1.0 -> velocity in km/s.
- Metrics (Table 7): Relative L2 (down), PSNR (up), SSIM (up), Data misfit (down).

Table-7 baselines AFDPS is compared against

- Traditional: Adam 0.333/9.97/0.305 ; Adam(dag) 0.089/21.27/0.679 ; LBFGS(dag) 0.070/23.40/0.704.
- PnP diffusion: DPS 0.250/14.11/0.491 ; LGD 0.244/12.29/0.341 ; DiffPIR 0.204/16.11/0.554 ; DAPS(dag) 0.201/14.91/0.321 ; PnP-DM 0.259/11.98/0.431 ; REDDiff 0.319/10.37/0.280. (Relative L2 / PSNR / SSIM. (dag) = initialized from blurred ground truth; AFDPS uses the prior-only, un-daggered regime. DDNM and PiGDM are absent -- they are linear-only.)

Troubleshooting

- Forward diverges / NaN: velocity left the CFL band. The operator clamps to [vel_min_kms, vel_max_kms]; lower guidance_gamma, raise num_steps, or tighten the clamp. (This is exactly how DAPS/PnP-DM fail in InverseBench Fig. 3; AFDPS's clamp + Tweedie evaluation are the guards.)
- All particles collapse to one (weight degeneracy): enable algorithm.method.sampler_kwargs.resample=true and/or raise num_particles; ESS is logged.
- relative-L2 ~ 1 (ignores data): sigma_y too large or guidance too weak -> lower problem.model.sigma_noise and/or guidance_gamma.
- ImportError for examples.seismic: Devito's seismic examples are not on the path -- install Devito from source or add its examples dir to PYTHONPATH (see README_InverseBench.md).
- ModuleNotFoundError 'afdps_fwi' or 'inverse_problems': run from cwd=navier_stokes/ via the provided scripts (they put both full_waveform_inversion/ and navier_stokes/ on sys.path).
- gn_hutchinson / guidance_mode=auto give odd traces: these use the best-effort Devito adjoint in afdps_fwi/devito_adjoint.py -- validate on-device, or use the default laplacian_mode=fd_divergence (exact, reuses the validated gradient).
- CUDA out of memory: lower algorithm.method.num_particles and/or problem.model.grad_chunk.

Key files & reference

- full_waveform_inversion/README.md -- the full report (formulation, gradient/Laplacian, wiring, tuning, GB200 setup, Table-7 template, caveats).
- full_waveform_inversion/afdps_fwi/{operator.py, sampler.py, algo.py, devito_adjoint.py} -- the FWI operator API, vendored sampler, inference wrapper, and Gauss-Newton adjoint probes.
- full_waveform_inversion/configs/{problem/fwi-afdps.yaml, algorithm/afdps.yaml, pretrain/fwi.yaml}.
- full_waveform_inversion/scripts/{run_fwi_afdps_single.sh, run_fwi_afdps_sharded.sh, run_fwi_afdps_gb200.sh, aggregate_fwi_afdps_results.py}.
- full_waveform_inversion/tests/test_sampler_cpu.py -- the Devito-free CPU verification.
- Reused harness: navier_stokes/{inverse_problems/acoustic.py (Devito forward+adjoint), eval.py (AcousticWave evaluator), training/dataset.py (LMDBData), README_InverseBench.md}.
- Papers (reference): TMLR_Rebuttal-main/6054_Solving_Inverse_Problems_.pdf (AFDPS) and 9220_InverseBench_Benchmarking.pdf (InverseBench; FWI = Sec. 3 + App. B.4 + Table 7).